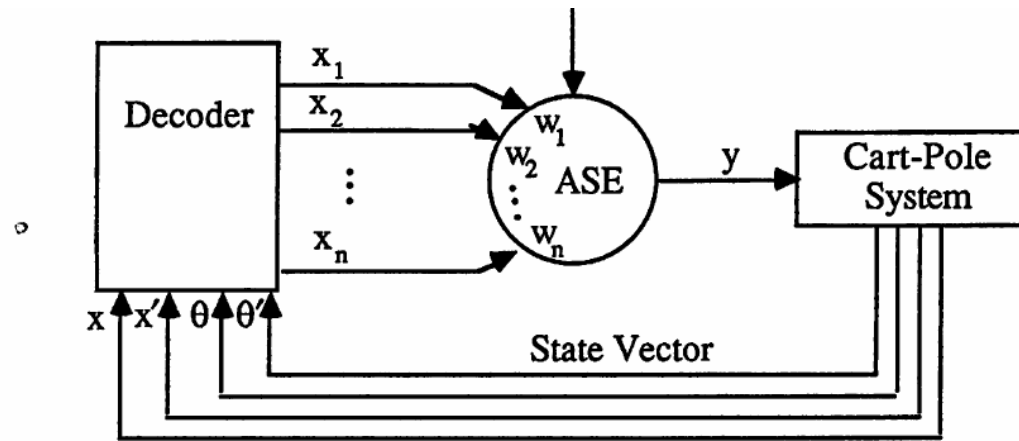


What the machine is actually doing

Figures and explanations from a tutorial by Jeff Smith,
written “late one night in November 1988.” Used with his permission.

1 THE MACHINE

A machine that taught itself to balance a pole



Barto, Sutton and Anderson's learning system, 1983.

The task: a cart on a track, a pole hinged to it. Push the cart left or right, keep the pole up, do not run off the end of the track.

That is the whole world this machine lives in, and learning to balance the pole is the only thing it has to do.

Nobody taught it how. That is the part worth slowing down on.

2 WHAT MADE IT HARD

Nobody tells it which part was wrong

No supervisor points at the mistake. The pole falls, and the machine alone has to work out which of its own actions deserves the blame.

That puzzle has a name: the credit assignment problem. Solving it is the whole jump: the earlier machine needed around 300 attempts, and once the machine could work out what went wrong, under 75.

Your instruments have the same problem: the sound is wrong, and you are doing eight things at once. Next: even once it knows what went wrong, it still needs a list of every situation it could be in.

Why you cannot just list every situation

Everyone before them chopped the world into boxes, one instruction per box. A couple of hundred boxes covered the pole — but the pole was a theoretical exercise, not a real machine.

Smith does the arithmetic for a real one — millimetre positions, real speeds, real angles — and gets about ninety million weights.

His reaction in 1988: “Hmmm....” Today’s large models carry hundreds of billions. Next: the feedback it learns from is thin, too.

The dog and the duck

Smith's explanation, and it is still the clearest one. A dog fetches a duck. Perfect retrieve: one biscuit. Soggy, sideways, half-dropped retrieve: the same biscuit.

One flat reward says nothing about the difference. What is missing is a sense of how it was going — expected it to go well and it did, small correction; expected it to go well and it did not, big correction.

So the machine grows a second half whose only job is predicting how well things are going. The gap between the prediction and what actually happened is the real teaching signal.

Blame, size, thin feedback — three problems, all from one cause: nobody ever tells it the answer.

Two ways to teach a machine

FLAIL — let it try, and say only whether it failed. It works, it costs hundreds of attempts, and somebody has to design the reward.

SHOW — hand it the movement and the sound you want, together. It never has to guess what you wanted, so it needs no boxes, no blame, and no critic.

Same ending either way: adjust the numbers inside until the wrongness is small.

So what exactly are those numbers inside? That is the next three slides.

What a “neuron” actually is

$$y(t) = f \left[\sum_{i=1}^n w_i(t) x_i(t) + \text{noise}(t) \right]$$

y is what comes out, x are the inputs, w are the weights.

Every input multiplied by its own weight, added up, and squeezed through a curve. That is the entire object.

Smith is blunt about the squeezing: it is not critical. A hard yes-or-no, a soft S, or a straight line all work.

Fine — but why squeeze it through anything at all?

But the bend is the whole point



Multiplying and adding is a straight-line operation. Feed a straight line into another straight line and you get a straight line. Stack fifty and it is still a straight line — every layer after the first would be pointless.

The curve bends the number on its way out. That bend is the only source of curvature in the machine, and it is why a little more tilt can open everything up instead of the sound just tracking your wrist.

The shape is fixed. So when it learns, what is actually changing? Only the weights.

Learning is nudging the weights

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha r(t) \mathbf{e}_i(t)$$

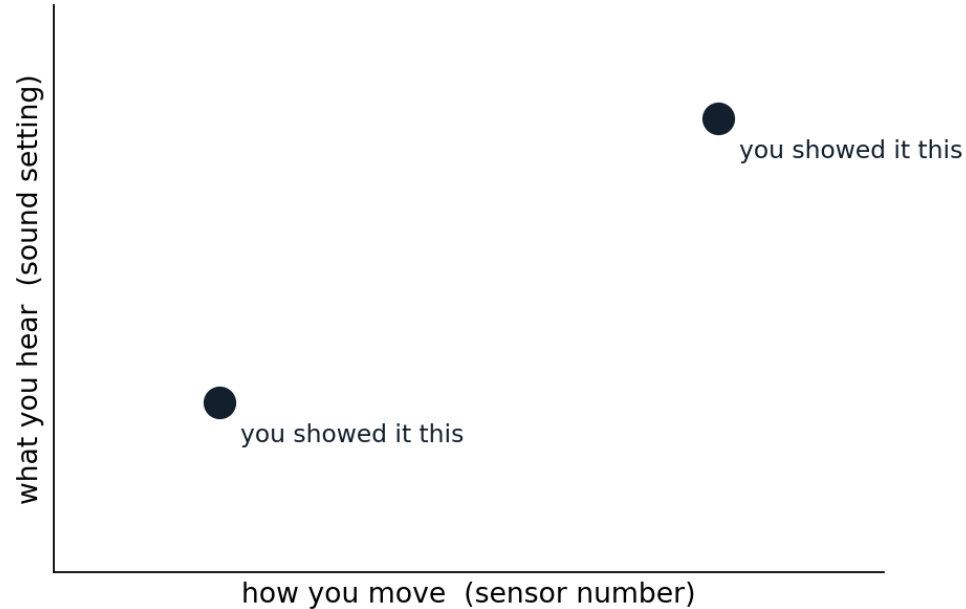
New weight = old weight, plus a small step in the direction that was less wrong.

Alpha is how big that step is: the learning rate, the same knob every one of these systems has. Too big and it never settles, too small and you wait all day.

That is the entire machine. Now put your hands on it.

9 NOW WITH YOUR HANDS

You demonstrate. That is one point.

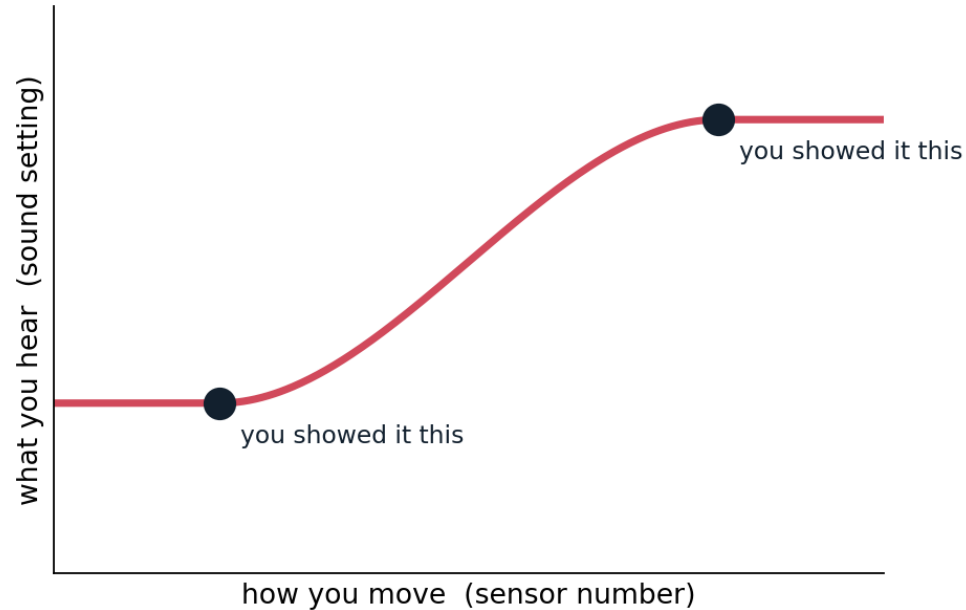


Hold a pose, set the sound you want, press record. The sensor numbers are the input. The sound settings are the answer you just supplied.

One dot on a graph. Now give it a second one.

10 NOW WITH YOUR HANDS

It fits a curve through your points



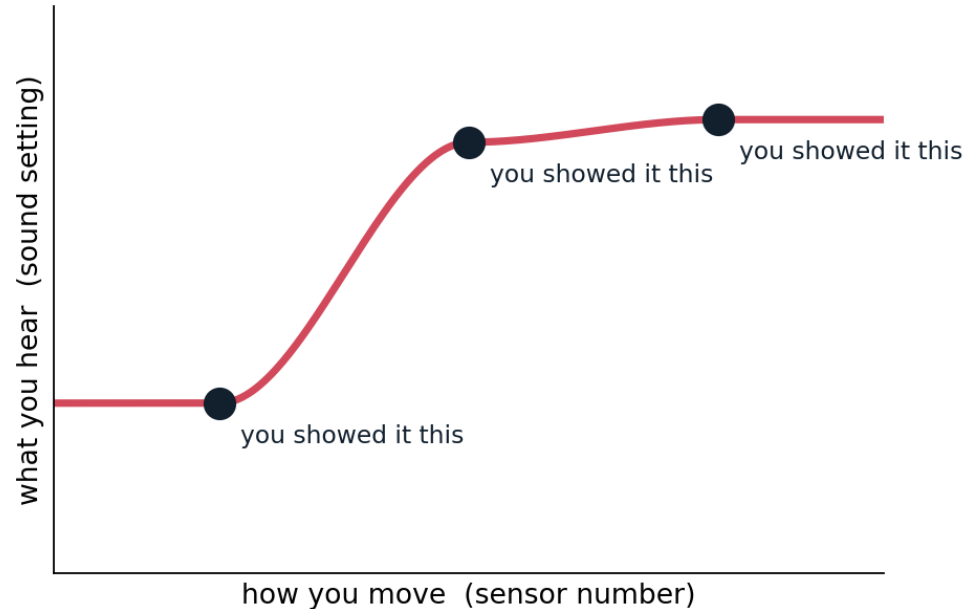
Two demonstrations, and every position in between now has a sound. You never specified those. They were invented.

That invented in-between is the instrument. It is the part you play.

Do not like what is happening in the middle? Do not edit it — show it another example.

11 NOW WITH YOUR HANDS

Add a demonstration, the curve re-bends



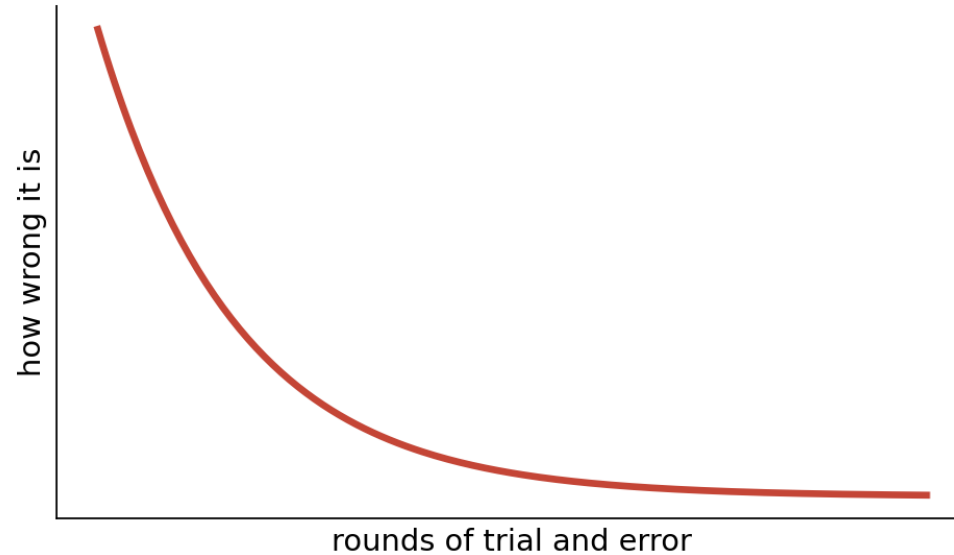
Something wrong in the middle? You do not go and edit numbers. You show it one more example and the whole curve re-fits around it.

Which is why this is called demonstrating rather than programming.

Each re-fit is a round of guess, measure, nudge. How long does that take?

12 HOW LONG IT TAKES

Repeat until close enough



Measured on the ES3C28P board, 25 September 2026. A second trainer solves it in one shot in 1.5 ms, by a different method.

Guess, measure how wrong, nudge, repeat. That loop is the whole of training.

On your board, twenty demonstrations take about thirteen seconds. The sketches train in slices, so the screen stays alive while it works.

Long enough to notice, short enough to retrain between takes. Last question: when should you reach for this at all?

So what is this good for

Smith asked in 1988 whether anyone could make money off this. Not yet, was the honest answer. Ours is a concert, which is a better deliverable anyway.

Anything you could write by hand as a rule — write it by hand. Clearer, faster, easier to fix.

Save the learning for what you cannot write down: several sensors at once, several sound controls, messy numbers underneath. Group 2: motion that drifts and shakes. Group 1: how two people move in relation to each other.

Worth arguing about: how few demonstrations can you get away with, and what is happening where you never showed it anything?

Where this came from

Michie & Chambers, “Boxes: An Experiment in Adaptive Control”

Barto, Sutton & Anderson, “Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problems” (1983)

Barto, Sutton & Anderson, “Synthesis of Nonlinear Control Surfaces by a Layered Associative Search Network” (1982)

Sutton & Selfridge, “A Model for Motor Control and Learning” · Connell & Utgoff, “Learning to Control a Dynamic Physical System” (1986)